

Python Backend Developer 10-Week Roadmap

For CS Students with React/JS Experience

■ ~2 hrs/day | ■ 10 Weeks | ■ Build & Deploy a Full REST API
with Database · Authentication · Tests · Docker

■ HIGH PRIORITY

■ MEDIUM

■ NICE TO KNOW

■ Interview Hot

■ JS → Python Shift

PHASE 1 — CORE PYTHON (Weeks 1–4)

Week 1 — Python Fundamentals (The JS → Python Bridge)

■ HIGH PRIORITY | ■ Interview Hot

■ Topics Covered

- Basic syntax & variables
- Data types
- Control flow: if / for / while
- Functions & return values

■ JS → Python Quick Shifts

JavaScript	Python
let, const, var	No keyword — just <code>x = 5</code>
{ curly braces }	Indentation IS the syntax
=== strict equality	== already works properly
for (let i = 0; ...)	for item in list: (direct)
null / undefined	Only None
true / false	True / False (capitalised)
console.log()	print()
function foo()	def foo():
=> arrow functions	lambda (single expression)
typeof x	type(x) or isinstance(x, int)

■ What to Practice

- Write 10 small functions (fibonacci, palindrome, FizzBuzz ...)
- Use `type()` and `isinstance()` constantly — build the habit
- Understand truthy/falsy: `[]`, `0`, `"`, `None` are all falsy

■ Mini-Project: CLI Calculator — takes user input, handles invalid input gracefully

Week 2 — Data Structures & Comprehensions

■ HIGH PRIORITY | ■ Interview Hot

■ Topics Covered

- Lists, tuples, dicts, sets
- List / dict / set comprehensions
- Iterators & enumerate

■ JS → Python Quick Shifts

JavaScript	Python
Array	list — similar with more built-in methods
No tuple	tuple — immutable, usable as dict key
Object / Map	dict — ordered since Python 3.7
Set	set — same concept, different syntax
.map() / .filter()	List comprehensions (more Pythonic)
arr.forEach((val,i)=>...)	for i, val in enumerate(arr):
Spread ...arr	Unpacking *list and **dict

■ What to Practice

- Rewrite 5 JS .map()/.filter() chains as comprehensions
- Use enumerate everywhere instead of manual counters
- Practice safe dict access with .get() (like ?. in JS)

■ **Mini-Project: Contact Book CLI — store contacts in dict, search/filter with comprehensions, save to JSON**

Week 3 — Functions Deep Dive & Pythonic Patterns

■ HIGH PRIORITY | ■ Interview Hot

■ Topics Covered

- Generators & yield
- Lambdas & higher-order functions
- Decorators
- Functional programming basics

■ JS → Python Quick Shifts

JavaScript	Python
No equivalent	yield — pauses function, returns generator
(x) => x * 2	lambda x: x * 2 (single expression only)
Higher-order functions exist	Same, but comprehensions preferred
No built-in decorator syntax	@decorator — syntactic sugar for wrapping
Closures work similarly	Closures similar; use nonlocal to modify

■ What to Practice

- Write a generator that reads a large file line by line
- Build 3 custom decorators: timing, logging, retry
- Understand decorator stacking: @a @b def f means a(b(f))

■ **Mini-Project: Build @retry(max_attempts=3) and @timer decorators; apply to API-calling functions**

Week 4 — Error Handling, File I/O, OOP & Environment Setup

■ HIGH PRIORITY | ■ Interview Hot

■ Topics Covered

- Exceptions & error handling
- File I/O, JSON & CSV
- Virtualenv / venv / pip
- OOP — classes, inheritance

■ JS → Python Quick Shifts

JavaScript	Python
try/catch/finally	try/except/else/finally (else = no exception)
throw new Error()	raise ValueError() — specific types
fs.readFile (async)	open() with with statement (context manager)
npm / package.json	pip / requirements.txt or pyproject.toml
node_modules	venv — virtual environment per project
class X extends Y	class X(Y): — multiple inheritance OK
this	self — explicitly passed, not implicit
constructor()	__init__() — one of many dunder methods

■ What to Practice

- Read/write JSON and CSV files
- Create a class hierarchy with inheritance
- Set up a project with venv from scratch
- Write custom exceptions for your project

■ **Mini-Project: Expense Tracker** — OOP-based classes, reads/writes CSV, handles errors, uses venv

PHASE 2 — INDUSTRY TOOLS & PRACTICES (Weeks 5–7)

Week 5 — REST API Development with FastAPI

■ HIGH PRIORITY | ■ Interview Hot

■ Topics Covered

- FastAPI basics
- Type hints & Pydantic models
- Environment variables & config
- Logging & debugging

■ JS → Python Quick Shifts

JavaScript	Python
app.get('/users', handler)	@app.get('/users') decorator
req.body	Pydantic model auto-parsed
req.params.id	Path param: def get_user(id: int):
req.query	Query parameter with defaults
.env + dotenv	.env + python-decouple or pydantic-settings
console.log debugging	logging module + pdb debugger
No built-in validation	Pydantic validates everything automatically

■ What to Practice

- Build full CRUD endpoints for a resource
- Use Pydantic models for request validation
- Return proper errors with HTTPException
- Add structured logging with Python's logging module

■ **Mini-Project: Task Manager API v1 — full CRUD, Pydantic validation, env config, structured logging**

Week 6 — Database Integration & Authentication

■ HIGH PRIORITY | ■ Interview Hot

■ Topics Covered

- SQLAlchemy ORM + PostgreSQL
- JWT authentication & authorization
- Third-party API calls (requests library)

■ JS → Python Quick Shifts

JavaScript	Python
Prisma / Sequelize	SQLAlchemy (most popular ORM)
jsonwebtoken	PyJWT or python-jose
fetch / axios	requests (sync) or httpx (async)
Passport.js	Manual JWT + FastAPI dependencies

■ What to Practice

- Connect Week 5 API to PostgreSQL via SQLAlchemy
- Add user registration & login endpoints
- Protect routes with JWT auth
- Call a third-party API (weather, quotes ...) — NEVER store plain-text passwords

■ **Mini-Project: Task Manager API v2 — PostgreSQL, user auth (register/login/protected), external API**

Week 7 — Testing & Code Quality

■ HIGH PRIORITY | ■ Interview Hot

■ Topics Covered

- pytest — unit tests, fixtures, mocking
- Code quality: ruff, black, mypy, type hints
- Git best practices
- Design patterns (Repository, Factory, Singleton)

■ JS → Python Quick Shifts

JavaScript	Python
Jest	pytest (dominant framework)
jest.mock()	unittest.mock / pytest-mock
ESLint + Prettier	ruff (linter) + black (formatter)
TypeScript compiler	mypy (static type checker)
.gitignore for node_modules	.gitignore for venv, __pycache__, .env

■ What to Practice

- Write unit tests for all API endpoints
- Write integration tests with a test database
- Mock external API calls in tests
- Set up pre-commit pipeline: black → ruff → mypy → pytest

■ Mini-Project: Full test suite for Task Manager API — unit + integration tests, 80%+ coverage

PHASE 3 — DEPLOYMENT & ADVANCED TOPICS (Weeks 8–10)

Week 8 — Docker & Deployment

■ MEDIUM PRIORITY | ■ Interview Hot (backend roles)

■ Topics Covered

- Docker basics for Python apps
- Deployment — Railway / Render / AWS
- Concurrency: threading, multiprocessing, async/await

■ JS → Python Quick Shifts

JavaScript	Python
Dockerfile (Node)	Dockerfile (Python) — similar structure
Vercel / Netlify	Railway / Render (for APIs)
async/await (single thread)	asyncio — same concept
No equivalent	threading — limited by GIL
Worker threads	multiprocessing — true parallelism

■ What to Practice

- Dockerize your Task Manager API
- Create docker-compose.yml with API + PostgreSQL
- Deploy to Railway or Render
- Add one async endpoint that makes concurrent API calls

■ Mini-Project: Dockerize & deploy Task Manager API — live on the internet with a public URL

Week 9 — Capstone Project: LinkVault

■ HIGH PRIORITY

■ Topics Covered

- Full project combining everything from Weeks 1–8

■ What to Practice

- User auth (JWT) + full CRUD for bookmarks with categories/tags
- Auto-fetch page title & description from URLs (requests + BeautifulSoup)
- Search & filter, rate limiting, background tasks for URL validation
- Full test suite (80%+ coverage), Dockerized, deployed & live

■ Mini-Project: LinkVault API — Bookmark Manager built entirely from scratch

Week 10 — Polish, Portfolio & Interview Prep

■ HIGH PRIORITY | ■ Interview Hot

■ Topics Covered

- Clean up capstone code
- Write a proper README
- Review all Interview Hot topics
- Do 2–3 mock interviews

■ Mini-Project: 2–3 mock interviews explaining your capstone architecture

■ Top 15 Python Interview Questions to Prepare

- | | |
|----|--|
| 1. | Mutable vs Immutable types — list vs tuple, why it matters |
|----|--|

2.	List vs Generator — memory efficiency, lazy evaluation
3.	Decorators — write one from scratch, explain stacking
4.	GIL — what it is, when it's a problem, how to work around it
5.	*args and **kwargs — explain and use in a function
6.	List comprehension vs map/filter — when to use which
7.	__init__ vs __new__ — object creation lifecycle
8.	Context managers — with statement, write a custom one
9.	is vs == — identity vs equality
10.	Shallow vs deep copy — copy() vs deepcopy()
11.	REST API design — status codes, resource naming, pagination
12.	SQL vs NoSQL — when to use which
13.	JWT flow — access vs refresh tokens, where to store them
14.	Docker basics — Dockerfile, layers, volumes, compose
15.	Testing strategy — unit vs integration vs e2e, what to mock

■ Weekly Summary at a Glance

Week	Focus	Priority	Mini-Project
Wk 1	Syntax, types, control flow, functions	■ HIGH	CLI Calculator
Wk 2	Data structures, comprehensions, iterators	■ HIGH	Contact Book CLI
Wk 3	Generators, decorators, functional patterns	■ HIGH	Custom Decorators
Wk 4	Errors, files, OOP, venv	■ HIGH	Expense Tracker
Wk 5	FastAPI, type hints, config, logging	■ HIGH	Task Manager v1
Wk 6	Database, auth, third-party APIs	■ HIGH	Task Manager v2
Wk 7	Testing, code quality, design patterns	■ HIGH	Test Suite + CI
Wk 8	Docker, deployment, concurrency	■ MEDIUM	Deploy Task Manager
Wk 9	Capstone project	■ HIGH	LinkVault API
Wk 10	Polish, portfolio, interview prep	■ HIGH	Mock Interviews

■ Tools You'll Use Daily

Tool	Purpose	JS Equivalent
python3	Runtime	node
pip	Package manager	npm
venv	Virtual environment	node_modules
pytest	Testing	Jest
black	Code formatter	Prettier
ruff	Linter	ESLint
mypy	Type checker	TypeScript compiler
uvicorn	ASGI server	nodemon
docker	Containerisation	Same
htpie / curl	API testing	Postman / fetch

■ Final Tips

- **Don't re-learn what you know.** You already understand loops, functions, and OOP. Focus on how Python does things differently.
- **Build, don't just read.** Each week has a project for a reason. Watching tutorials without coding teaches you nothing.
- **Use type hints from Day 1.** It makes your code readable and prepares you for FastAPI and mypy.



Write Pythonic code. Follow PEP 8, use meaningful names, write docstrings. 'Pythonic' style matters in interviews.



Your React background is an asset. You already understand component architecture and async patterns. Python backend is the missing piece.

Good luck! ■ Build consistently, ship something every week, and you'll be interview-ready in 10 weeks.